# CONSTRAINT-BASED PROBLEM SOLVING

## Answers for Questions 31, 32 and 33

**Name:** N. PARNITA

**Register No.:** 192525322

**Course code:** CSA0617

**Course Name:** Design and Analysis of Algorithm

## Q31. Steiner Tree Problem

**Problem Type: NP-Hard / Constraint Optimization Problem**

### Problem Definition

Given a weighted graph and a set of required terminal vertices, the Steiner Tree Problem finds a minimum-cost tree connecting all required terminals. Additional non-terminal vertices may be included when they reduce the total cost.

### Constraints

- All required terminal vertices must be connected.
- The selected edges must not form a cycle.
- The total edge cost must be minimized.
- Extra Steiner vertices may be used when beneficial.

### Backtracking Approach

The solution is built by selecting or rejecting edges recursively. A partial solution is checked for validity, and branches whose cost is already greater than the best solution found are pruned.

### Python Code

```
def steiner_search(edges, terminals):
    best_cost = float("inf")
    best_tree = []

    def search(i, chosen, cost):
        nonlocal best_cost, best_tree

        if cost >= best_cost:
            return

        if i == len(edges):
            if all_terminals_connected(chosen, terminals):
                best_cost = cost
                best_tree = chosen[:]
            return

        search(i + 1, chosen + [edges[i]], cost + edges[i][2])
        search(i + 1, chosen, cost)

    search(0, [], 0)
    return best_tree, best_cost
```

```
    # all_terminals_connected() checks connectivity
    # among the required terminal vertices.
```

### Result

The backtracking method explores feasible edge combinations and keeps the minimum-cost tree. Pruning reduces unnecessary searches. Since Steiner Tree is NP-Hard, the method is mainly practical for small graphs.

## Q32. Maximum Cut Problem

**Problem Type: NP-Hard Problem**

### Problem Definition

The Maximum Cut Problem divides the vertices of a graph into two sets so that the number or total weight of edges connecting the two sets is as large as possible.

### Constraints

- Every vertex belongs to exactly one of the two sets.
- An edge contributes to the cut only when its endpoints are in different sets.
- The total cut value must be maximized.

### Backtracking Approach

Each vertex is recursively assigned to Set 0 or Set 1. After all assignments are made, the cut value is calculated. A branch-and-bound upper bound can be used to prune partitions that cannot improve the current best solution.

### Python Code

```
def max_cut(graph):
    n = len(graph)
    side = [0] * n
    best_value = 0
    best_side = None

    def value():
        total = 0
        for u in range(n):
            for v in range(u + 1, n):
                if side[u] != side[v]:
                    total += graph[u][v]
        return total

    def backtrack(v):
        nonlocal best_value, best_side

        if v == n:
            cur = value()
            if cur > best_value:
                best_value = cur
                best_side = side[:]
            return

        side[v] = 0
        backtrack(v + 1)
```

```
            side[v] = 1
            backtrack(v + 1)

        backtrack(0)
        return best_value, best_side
```

### Result

The algorithm examines the possible two-set partitions and returns the partition with the largest cut value. Backtracking guarantees the optimal result for a small graph because all assignments are considered.

## Q33. Boolean Circuit Satisfiability

**Problem Type: NP-Complete Problem**

### Problem Definition

The problem determines whether there is a True/False assignment to the circuit's input variables that makes the final circuit output True.

### Constraints

- Each input variable receives True or False.
- AND requires all inputs to be True.
- OR requires at least one input to be True.
- NOT reverses its input value.
- The final circuit output must be True.

### Backtracking Approach

Boolean values are assigned one variable at a time. The circuit is evaluated after assignments are made. If a partial assignment cannot lead to a True output, that branch can be pruned. A satisfying assignment is returned when the final output becomes True.

### Python Code

```
def circuit_sat():
    assignment = [False] * 3

    def evaluate():
        x0, x1, x2 = assignment
        return (x0 and x1) or (not x2)

    def backtrack(i):
        if i == 3:
            return evaluate()

        assignment[i] = False
        if backtrack(i + 1):
            return True

        assignment[i] = True
        if backtrack(i + 1):
            return True

        return False
```

```
    return backtrack(0), assignment

print(circuit_sat())
```

## Sample Input

```
Circuit: (x0 AND x1) OR (NOT x2)
```

## Sample Output

```
Satisfiable: True
```

## Result

The algorithm determines whether at least one input assignment satisfies the circuit. Backtracking is complete because it can examine all Boolean assignments, while constraint checking can reduce unnecessary exploration. The problem is NP-Complete.

# Conclusion

Steiner Tree and Maximum Cut are NP-Hard optimization problems, while Boolean Circuit Satisfiability is an NP-Complete decision problem. In all three cases, backtracking provides a systematic way to explore solutions, and pruning improves efficiency by eliminating unsuitable branches.